

Laboratórne cvičenie 3

Playlist a vlastná knižnica

Pokročilé objektové technológie

100 minút

Vytvoríte konzolovú aplikáciu na správu **hudobného playlistu**, teda poradia skladieb pre večierok, cestu vlakom alebo tréning. Vytvoríte vlastnú Class Library a Console App a vložíte dodané zdroje. Preskúmate iterátory, udalosti a filter s lambdou. Potom navrhnete vlastnú funkciu aplikácie a jej testy. Pri práci môžete používať GitHub Copilot alebo iného AI asistenta.

Aplikácia uchováva názvy, interpretov a trvanie skladieb. Jej textové menu umožní skladbu pridať, odobrať alebo zobrazíť poradie. Hudobné súbory neprehráva. Údajové štruktúry už poznáte; tu ich využijete na pochopenie vlastností C# a spolupráce projektov.

Čo je základ a čo je rozšírenie

Základ hodiny je U1 až U5 a vlastná funkcia s aspoň tromi testami. Označené metódy sú spoločný základ pre experimenty. Po ich doplnení ešte samostatne rozhodnete o návrhu vlastnej funkcie, zapojíte ju do menu a overíte hraničné situácie. NuGet tabuľka a balenie knižnice sú dobrovoľné.

- Získate vlastnú knižnicu použiteľnú pre skladby aj iné typy.
- Preskúmate yield, MoveNext, lambdu a odložené vyhodnotenie.
- Prihlásite odberateľa udalosti, sledujete zmeny a odberateľa odhlásite.
- Navrhnete vlastné API a testy, ktoré zachytia chybu jeho správania.

Orientačný plán na 100 minút

Minúty	Blok	Kontrolný výsledok
0–15	U1 Vlastné projekty a zdroje	Build a pripravená ukážka
15–30	U2 Krátke doplnenie operácií	Poradie a kapacita
30–50	U3 Iterátory	Dopredný a spätný priechod
50–60	U4 Udalosť	Odberatelia a odhlásenie
60–75	U5 Lenivý filter	Lambda a čas vyhodnotenia
75–95	Vlastná funkcia a testy	Nový príkaz a tri vlastné testy
95–100	Rezerva alebo rozšírenie	Dokončenie alebo NuGet

Pracujte vlastným tempom. Riešenie sa neodovzdáva. Ak ste bez posunu, použite Pomoc na konci návodu. Pri technickom probléme použite záložný starter; vytváranie projektov Vás nemá zastaviť.

Na Windows postupujte podľa strany Vlastné projekty vo Visual Studiu. Na macOS alebo Linuxe použite nasledujúci postup cez CLI. Zdroje C# sú spoločné, mení sa iba ovládanie IDE.

U1 Vlastné projekty vo Visual Studiu 2026

0–15 minút • Windows • macOS a Linux použijú postup cez CLI

Rozbaľte **POT_Lab03_Zdroje_NaVytvorenieProjektov.zip** do priečinka s podkladmi. Obsahuje C# zdroje knižnice a aplikácie, pripravený testovací projekt a stručný README. Modely, menu a register skladieb sú hotové; označené metódy doplníte. Core.csproj, App.csproj ani solution v balíku nie sú, vytvoríte ich sami.

Vytvorte riešenie a knižnicu

1. Na úvodnej obrazovke Visual Studia vyberte **Create a new project**. Ak už máte otvorené iné riešenie, použite **File > New > Project**.
2. Vyhľadajte **Class Library**, vyberte jazyk **C#** a modernú šablónu bez dodatku .NET Framework. Kliknite **Next**.
3. Do **Project name** zadajte **PlaylistLab.Core**, do **Solution name** zadajte **PlaylistLab** a vyberte **Location** pre svoje projekty. Voľbu **Place solution and project in the same directory** nechajte vypnutú, aby mala knižnica vlastný podpriečinok. Kliknite Next, vyberte **.NET 10.0** a Create.
4. Otvorte **View > Solution Explorer**, ak ho nevidíte. Uvidíte solution PlaylistLab a v nej Core. Solution je obal pre súvisiace projekty; Visual Studio ju vytvorilo spolu s knižnicou. Blank Solution tu nepotrebuje.

Knižnica sa prekladá do súboru **DLL**. Obsahuje triedy, ktoré používa iný projekt; nemá vlastné konzolové spustenie. Označenie Core tu znamená spoločný kód nezávislý od používateľského menu.

Vytvorte aplikáciu a referenciu

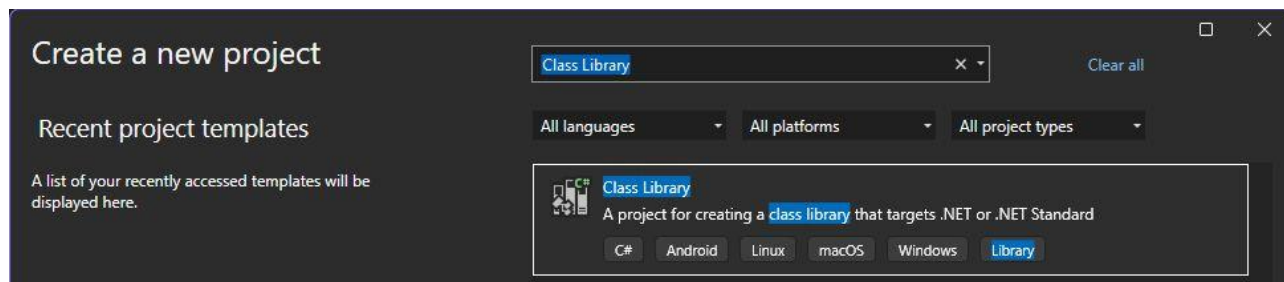
5. Na solution opäť použite **Add > New Project**. Vyberte C# **Console App**, kliknite Next, zadajte **PlaylistLab.App**, ponechajte Location v priečinku riešenia a kliknite Next. Vyberte **.NET 10.0** a Create. Ostatné voľby ponechajte predvolené.
6. Rozbaľte App. Na **Dependencies** kliknite pravým tlačidlom a vyberte **Add Project Reference**. V zozname projektov označte **PlaylistLab.Core** a potvrdte **OK**. Pod **Dependencies > Projects** má byť Core.
7. Na App kliknite pravým tlačidlom a zvolte **Set as Startup Project**. Pri spustení sa vykoná jej Program.cs, ktorý môže používať verejné typy z knižnice.

Referenciu pridávate **z App na Core**. Core neodkazuje na App. Project Reference prepája projekty aj pri preklade; DLL nekopírujete ručne.

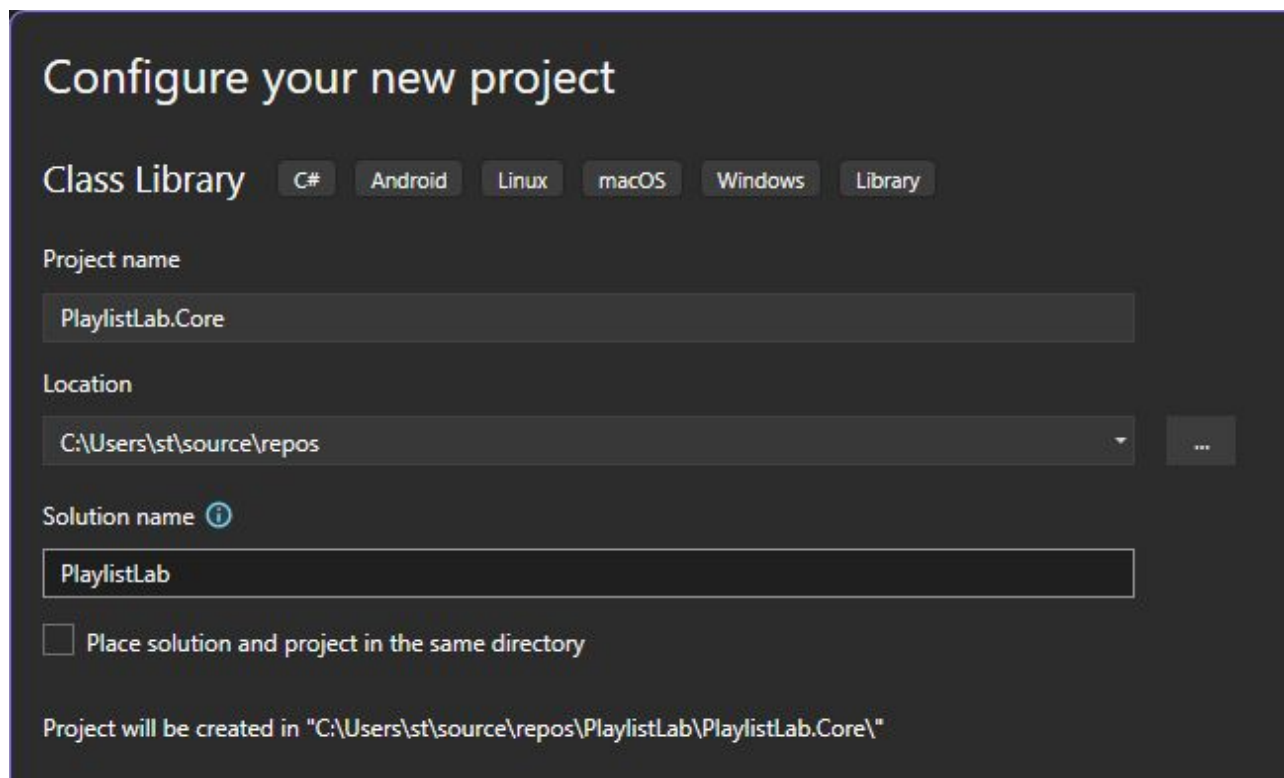
Solution súbor je PlaylistLab.slnx alebo PlaylistLab.sln. Ak máte .sln, v CLI príkazoch návodu nahraďte iba túto príponu. Pokračujte časťou U1 Vloženie zdrojov do vlastných projektov.

U1 Obrazová orientácia pri vytváraní

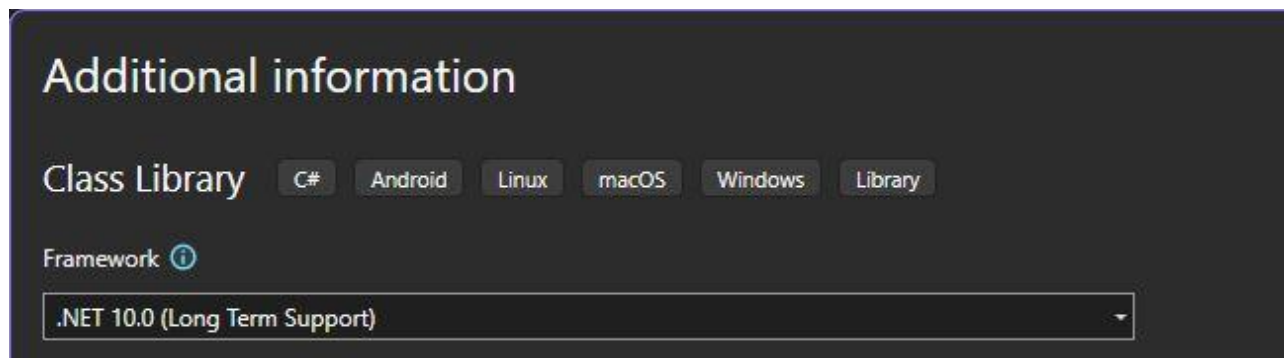
Skutočné výrezy z Visual Studio 2026 na Windows. Farby či usporiadanie sa môžu líšiť. Sú doplnkom postupu; na macOS a Linuxe pokračujte cez CLI.



1. **Šablóna:** Class Library s jazykom C# a podporou viacerých platforiem.



2. **Názvy:** PlaylistLab.Core a PlaylistLab. Location si vyberte podľa seba; voľbu spoločného priečinka nechajte vypnutú.



3. **Framework:** .NET 10.0. Potom stlačte Create podľa textového postupu.

U1 Vloženie zdrojov do vlastných projektov

V Solution Exploreri kliknite pravým tlačidlom na solution a použite **Open Folder in File Explorer**. Nájdite priečinok obsahujúci solution súbor. Vedľa neho majú byť priečinky PlaylistLab.Core a PlaylistLab.App. To je **koreň riešenia**; odtiaľ neskôr spúšťate CLI príkazy.

Skopírujte obsah podkladov

V druhom okne Prieskumníka otvorte rozbalené podklady. Kopírujte obsah do už existujúcich projektových priečinkov podľa tabuľky. Nevytvárajte vnorenie PlaylistLab.Core/PlaylistLab.Core.

Podklady	Kam ich vložiť
Z PlaylistLab.Core priečinky Collections Models Services	Do vlastného PlaylistLab.Core
Z PlaylistLab.App štyri súbory s príponou .cs	Do vlastného PlaylistLab.App; potvrdte nahradenie Program.cs
Celý priečinok PlaylistLab.Tests	Vedľa Core a App

1. Vo vlastnom Core odstráňte **Class1.cs**, ktorý vytvorila šablóna. Vlastné Core.csproj a App.csproj ponechajte. Po návrate do Visual Studia sa skopírované .cs súbory zobrazia automaticky; netreba ich pridávať po jednom.
2. Zapnite XML dokumentáciu Core jedným z dvoch spôsobov. Vo Visual Studiu otvorte **Properties > Build > Output** a označte **Generate a file containing API documentation**. Alebo použite **Edit Project File**, pred existujúce `</PropertyGroup>` vložte riadok nižšie a uložte Ctrl+S. Obe možnosti zapnú tvorbu XML dokumentácie k DLL.

```
<GenerateDocumentationFile>true</GenerateDocumentationFile>
```

3. Na solution použite **Add > Existing Project**, otvorte priečinok PlaylistLab.Tests a vyberte **PlaylistLab.Tests.csproj**. Potvrdte Open. Testovací projekt je pripravená pomôcka na overenie, nemusíte vytvárať jeho testy ani nastavovať balíky.
4. Vyberte **Build > Build Solution**. Ak sa otvorí **Error List**, riešte najprv položky typu Error. Pri chýbajúcom type z Core skontrolujte referenciu v App a správne umiestnenie súborov. Pri prvom preklade sa automaticky obnovia testovacie balíky.

V Solution Exploreri teraz vidíte **tri projekty**: Core s Collections, Models a Services, App so štyrmi .cs súbormi a Tests. Build má skončiť úspešne. Spoločné Directory.Build.props ani NuGet.Config nekopírujete; projekty majú vlastné nastavenia a používajú bežný zdroj nuget.org. PACKAGE_README.md je podklad pre dobrovoľné balenie, teraz ho nepotrebujete.

Ak sa príprava nepodarí do 15 minút: Rozbaľte POT_Lab03_Starter_NET10.zip do iného priečinka a cez **File > Open > Project/Solution** otvorte PlaylistLab.slnx. Pokračujte rovnakými úlohami. Stranu s CLI môžete na Windows preskočiť.

U1 Vlastné projekty cez CLI

macOS alebo Linux s VS Code či Riderom • rovnaká možnosť aj na Windows

Použite VS Code alebo Rider a terminál, s ktorými ste pracovali na Lab02. Vo VS Code s C# Dev Kit môžete používať aj Solution Explorer a testy priamo v IDE.

Vytvorte projekty

Nasledujúce príkazy sú rovnaké pre PowerShell, bash aj zsh. Spúšťajte každý riadok samostatne, v tomto poradí, v novom pracovnom priečinku:

```
mkdir moj-playlist
cd moj-playlist
dotnet new sln -n PlaylistLab --format slnx
dotnet new classlib -n PlaylistLab.Core -f net10.0 --no-restore
dotnet new console -n PlaylistLab.App -f net10.0 --no-restore
dotnet sln add PlaylistLab.Core/PlaylistLab.Core.csproj
dotnet sln add PlaylistLab.App/PlaylistLab.App.csproj
cd PlaylistLab.App
dotnet add reference ../PlaylistLab.Core/PlaylistLab.Core.csproj
cd ..
```

Vložte spoločné zdroje

Rozbaľte POT_Lab03_Zdroje_NaVytvorenieProjektov.zip do samostatného priečinka. Cez Finder alebo správcu súborov vložte jeho obsah podľa tabuľky na predchádzajúcej strane.

Odstráňte iba šablónový Core/Class1.cs, nahraďte App/Program.cs a skopírujte aj ďalšie tri App súbory. Do existujúceho PropertyGroup vlastného Core.csproj pridajte

<GenerateDocumentationFile>true</GenerateDocumentationFile>. Vlastné Core.csproj a App.csproj ponechajte. Dodaný testovací priečinok pridajte do solution a overte preklad:

```
dotnet sln add PlaylistLab.Tests/PlaylistLab.Tests.csproj
dotnet restore PlaylistLab.slnx
dotnet build PlaylistLab.slnx
```

Vo **VS Code** otvorte cez File > Open Folder koreň so solution. V **Rideri** otvorte PlaylistLab.slnx. Ak IDE solution neponúkne, pokračujte cez CLI; .NET 10 SDK tento formát používa. Na Linuxe dodržte veľkosť písmen názvov súborov a priečinkov.

Pri technickom probléme použite záložný starter. V ďalších kapitolách sú Core a App skratky rovnomenných projektov. Klávesové skratky opisujú Visual Studio; v inom IDE použite zodpovedajúce Run, Debug a Step Over.

Práca v IDE a AI asistenti

Overte štart a spúšťajte iba aktuálny blok

Vo Visual Studiu 2026 otvorte **Test > Test Explorer** a zvolte **Run All**. Časť testov je zatiaľ červená, pretože označené metódy čakajú na doplnenie. Po každom bloku do vyhľadávania napíšte napríklad **U2_**, označte zobrazené testy a použite **Run Selected Tests**. Sledujte výsledok práve riešeného bloku.

Pri App otvorte **Properties > Debug > Open debug launch profiles UI**.

Nastavte **--basic** v poli **Command-line arguments**, zavrite nastavenie a spustite **Debug > Start Without Debugging** alebo **Ctrl+F5**. U1 vypíše 10, 20, odstráni 20 a zobrazí album Bez albumu; overí aj pripravený register skladieb. Štartovací základ hlási **4 nedokončené bloky a 0 neočakávaných chýb**.

Pre konkrétny pokus argument zmeňte na **--unit U3**; pre menu na **--interactive**. Na ladenie použite **F5**, breakpoint vložte kliknutím do ľavého okraja editora. Step Over je **F10**; hodnoty sledujte v Locals alebo Watch. Po pokuse vráťte argument na **--basic**.

V Rideri zvolte run configuration pre PlaylistLab.App a argumenty v **Run > Edit Configurations**. Vo VS Code s C# Dev Kit otvorte **Program.cs** a použite **Run and Debug**; na interaktívne menu je najjednoduchší integrovaný terminál. V inom IDE nemusíte napodobňovať skratky Visual Studia.

Spoločné príkazy pre všetky systémy

Z koreňa solution môžete použiť rovnaký preklad, testy aj spustenie:

```
dotnet build PlaylistLab.slnx
dotnet test PlaylistLab.slnx --filter "Unit=U2"
dotnet run --project PlaylistLab.App -- --unit U2
dotnet run --project PlaylistLab.App -- --basic
```

Použite **U2**, nie samotné číslo 2. Pre vlastné menu slúži **--interactive**. Príkazy môžete vložiť do terminálu vo Visual Studiu cez **View > Terminal** alebo do terminálu svojho IDE.

Používajte AI asistenta podľa vlastného výberu

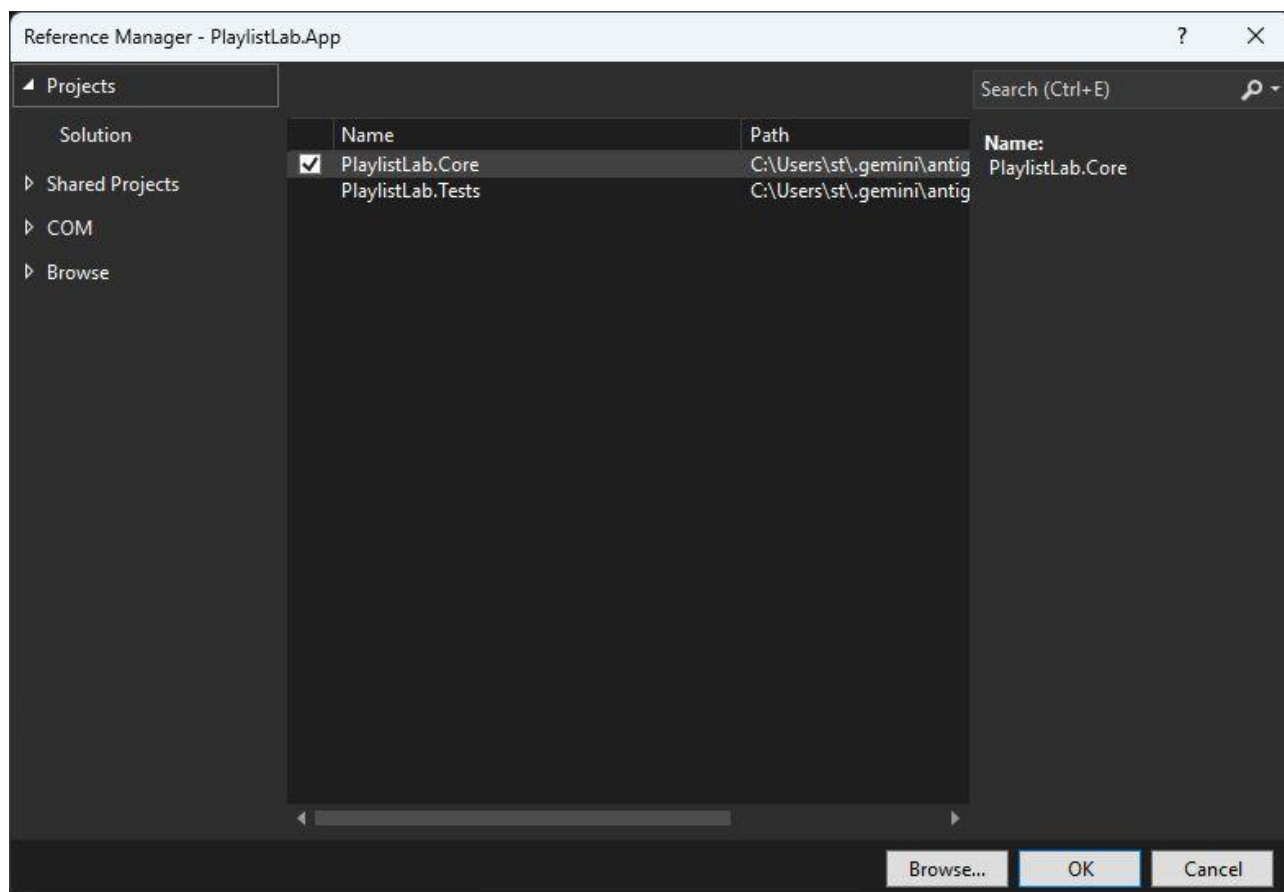
Môžete použiť **GitHub Copilot**, **Antigravity**, **Claude Code** alebo **ChatGPT** či **Codex**. Vo Visual Studiu je praktický integrovaný Copilot, iné nástroje používajte v prostredí, ktoré poznáte. Asistentovi sprístupnite relevantné zdroje. Po návrhu skontrolujte zmeny, spustite test bloku a vlastný pokus. Dodané testy nemeňte; svoje testy pridáte samostatne.

Prompty pri úlohách sú pomôcka pre ktorýkoľvek z týchto nástrojov. Najprv sami predpovedzte výsledok pokusu. Pri chybe pridajte presné hlásenie a požadované správanie. V záverečnej úlohe určíte kontrakt funkcie Vy; asistent môže pomôcť s návrhom, implementáciou aj testami.

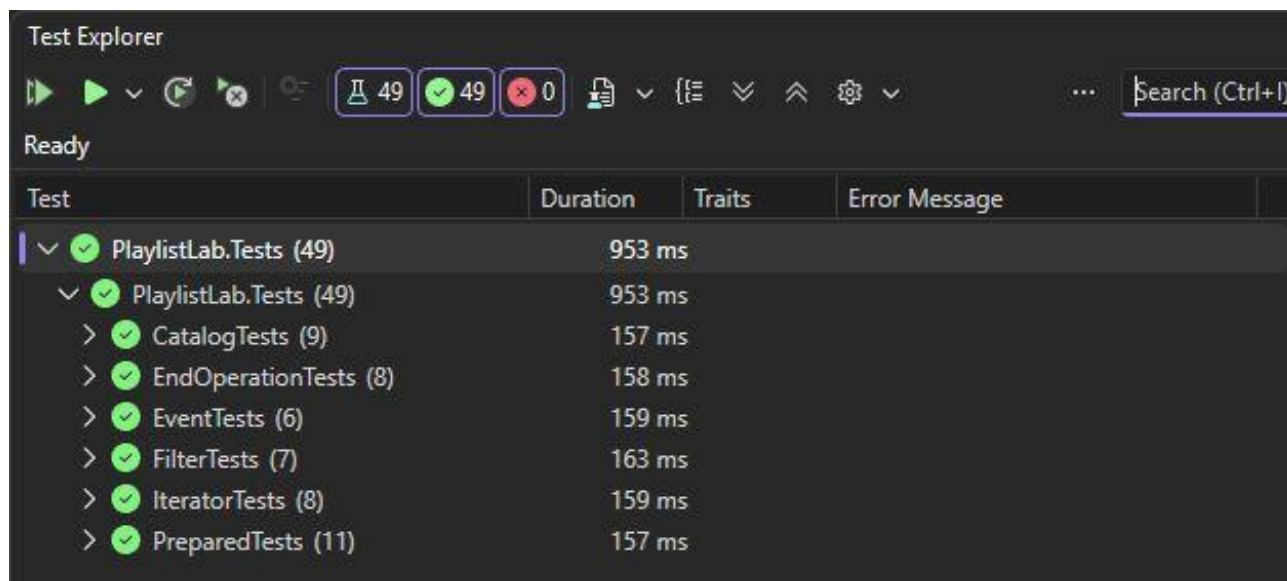
Pripravené časti: Track, uzol, indexer, AddFirst, RemoveLast, kapacita, verzia kolekcie, register ItemCatalog, menu a testy. **Vaša práca:** Vlastné projekty, U2 až U5, nová funkcia a jej testy. Dictionary v registri je hotová podporná časť. Ukážkové dáta v Program a SampleData ponechajte.

Referencia a testy vo Visual Studiu

Tieto skutočné screenshots dopĺňajú U1 a časť Práca v IDE. Nepridávajú ďalšiu úlohu.



4. Referencia: Dialóg patrí App. Označte Core a potvrdte OK; App nesmie odkazovať na Tests. Core sa tým prístupný aplikácii.



5. Test Explorer: Ukážka stavu po dokončení U5: 49 dodaných testov je zelených. Na začiatku časť testov zámerne zlyháva. Vlastné aspoň tri testy pridáte až pri záverečnej funkcii.

U2 Kapacita a operácie nad uzlami

15–30 minút

Prečo vlastný zoznam: Precvičíte API v C#, nullable uzly, generiká, iterátor a udalosť v jednej triede. V praxi vyberajte podľa potreby hotový `List<T>`, `Queue<T>` alebo `LinkedList<T>`; vlastnú kolekciu píšete až pri konkrétnom dôvode. Tu nebudujete hudobnú službu pre produkciu.

Súbor: `Core/Collections/DoublyLinkedList.cs`. Skratky `Core` a `App` označujú projekty `PlaylistLab.Core` a `PlaylistLab.App`. Doplňte iba telá medzi `BEGIN` a `END U2a` a `U2b`.

U2a Pridajte položku na koniec

Implementujte `AddLast`. Najprv zavolajte pripravené `EnsureSpace`. Pri plnej kolekcii vyvolá `InvalidOperationException` **pred zmenou uzlov, počtu a udalosti**.

Vytvorte nový `DoublyLinkedListNode` s hodnotou `item`. Jeho `Previous` bude pôvodný `_last`. Ak je zoznam prázdny, nastavte aj `_first` na nový uzol. Inak spojte pôvodný posledný uzol cez `Next` s novým. Aktualizujte `_last`, zvýšte `Count` a `_version` a zavolajte `OnChanged` s `Added`. Rovnaká skladba môže byť v playliste viackrát.

U2b Odoberte prvú položku

Implementujte `RemoveFirst`. Pri prázdnom zozname vyvolajte `InvalidOperationException` bez zmeny stavu. Uchovajte pôvodný prvý uzol a posuňte `_first` na jeho `Next`.

Ak po odobratí nič nezostáva, nastavte aj `_last` na `null`. Inak nastavte `Previous` nového prvého uzla na `null`. Odpojte `Next` odobratého uzla, znížte `Count`, zvýšte `_version`, oznámte `Removed` a vráťte hodnotu pôvodného uzla. Ako vzor štruktúry použite pripravenú `RemoveLast`.

Prompt: „Doplňte iba `U2a` a `U2b` v `DoublyLinkedList.cs` podľa návodu. Nepoužite `List` ani `LinkedList` namiesto pripravených uzlov. Zachovajte `EnsureSpace`, `Count`, `_version` a `OnChanged`. Vysvetlite prázdny zoznam, jednu položku a prepojenie `Next/Previous`. Neupravujte testy.“

Predpoveď a kontrola

V bloku `U2` aplikácie má zoznam kapacitu 3. Po pridaní 10, 20, 30 sa pridanie 40 odmietne. `RemoveFirst` vráti **10**, zostanú **20, 30** a počet bude **2**. Vyskúšajte následné vloženie ďalšej položky aj odobratie jedinej položky a opätovné použitie zoznamu.

Pri `AddLast` nastavte breakpoint na spojenie uzlov. Cez `F5` a `F10` sledujte `_first`, `_last`, `Count` a `Previous` nového uzla. Pridanie na koniec ani odobratie zo začiatku nemá prechádzať celý zoznam.

Kontrolný bod: Všetky `U2_` testy sú zelené. `U4` zatiaľ neoznamuje udalosti, preto jeho testy môžu stále zlyhávať. Pri zaseknutí použite `Pomoc` pre operácie nad uzlami.

Otázka pre seba: Prečo treba pri odstránení jedinej položky opraviť oba konce? Prečo je `foreach` vhodnejší na úplné prejdienie tohto zoznamu než `for` s indexerom, ktorý každý prvok hľadá od začiatku?

U3 Enumerátory a iterátory

30–50 minút

Súbor: Core/Collections/DoublyLinkedList.cs. GetEnumerator, negenerická implementácia a kontrola verzie sú pripravené. Doplňte U3a a U3b.

U3a Dopredné iterovanie

V EnumerateForward začnite na `_first`. Kým uzol nie je null, overte `EnsureUnchanged(expectedVersion)`, poskytnite jeho Value cez `yield return` a posuňte sa cez `Next`. Overenie verzie vykonajte aj **po skončení cyklu**.

U3b Spätné iterovanie

V EnumerateReverse použite rovnaký postup od `_last` a posúvajte sa cez `Previous`. Reverse má poskytovať postupnosť bez `ToList`, `poľa` a kopírovania položiek. Prázdny zoznam sa úspešne prejde bez položiek.

```
foreach (int value in numbers)
    Console.WriteLine(value);
foreach (int value in numbers.Reverse())
    Console.WriteLine(value);
```

Prompt: „Implementujte iba U3a a U3b cez `yield return` a pripravené `Next/Previous`. V každej iterácii aj na konci overte `EnsureUnchanged`. Nevytvárajte kópiu kolekcie. Vysvetlite rozdiel `IEnumerable` a `IEnumerator` a kedy sa telo iterátora začne vykonávať.“

Preskúmajte MoveNext v debuggeri

Spustite blok U3. Očakávajte **1, 2, 3** dopredu a **3, 2, 1** dozadu. Podľa nasledujúcej strany preskúmajte breakpoint na `yield return` a hodnoty v Locals. Pri prvom `MoveNext` sa kód vykoná po prvé `yield`; po jeho vykonaní bude `Current` 1. Ďalšie `MoveNext` pokračuje za predchádzajúcim `yield`. `Current` čítajte iba po úspešnom `MoveNext`.

Kompilátor vytvorí stav enumerátora aj metódy `MoveNext`, `Current` a `Dispose`. Manuálnu triedu `IEnumerator` tu nemusíte implementovať. Každé nové `GetEnumerator` vytvorí samostatný priechod. Reset generovaného iterátora nepoužívajte, pri ďalšom priechode získajte nový enumerátor.

Zmena zoznamu počas priechodu

Pripravená kontrola verzie odmietne zmenu zoznamu počas priechodu. Pozrite test `U3_MutationDuringEnumerationIsRejected` v `PlaylistTests.cs`: po získaní enumerátora zmení zoznam a overí výnimku pri ďalšom `MoveNext`. Toto pravidlo chráni priechod po uzloch. Vlastný podobný pokus je dobrovoľný.

Verzia sa v pripravenom `GetEnumerator` a `Reverse` zachytí už pri ich zavolaní. Zmena po získaní enumerátora alebo spätnej postupnosti ich zneplatní aj pred prvým `MoveNext`. Samotné `foreach` položky neodoberá; odobratie je samostatná operácia.

Kontrolný bod: U3_ testy sú zelené. Vysvetlite zastavenie na `yield` a pokračovanie pri ďalšom `MoveNext`. Pomoc pre iterátory obsahuje krátky funkčný vzor.

U3 Ladenie iterátora vo Visual Studiu 2026

Krátky pokus v rámci času U3 po doplnení iterátorov. Na macOS a Linuxe použite v Rideri alebo VS Code zodpovedajúce Breakpoint, Debug, Continue a panel premenných.

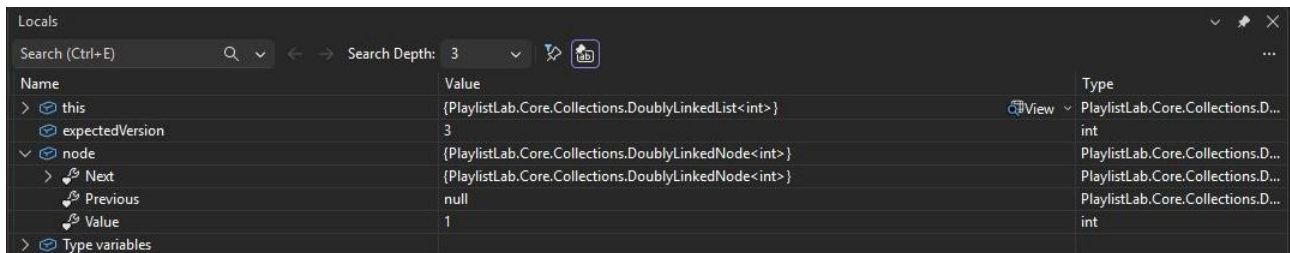
1. V nastavení spúšťania App zmeňte Command-line arguments na **--unit U3**. App musí byť Startup Project a konfigurácia **Debug**.
2. Otvorte Core/Collections/DoublyLinkedList.cs. V **EnumerateForward** nájdite `yield return node.Value;`. Kliknite do ľavého okraja tohto riadku alebo na riadku stlačte **F9**. Objaví sa červený breakpoint.
3. Spustíte **Debug > Start Debugging** alebo **F5**. Program sa zastaví na breakpointe. Žltá šípka a zvýraznenie označujú príkaz, ktorý sa ešte len vykoná. Číslo riadku sa môže líšiť podľa Vášho zápisu.



```
122 private IEnumerable<T> EnumerateForward(int expectedVersion)
123 {
124     // BEGIN U3a
125     var node = _first;
126     while (node is not null)
127     {
128         EnsureUnchanged(expectedVersion);
129         yield return node.Value;
130         node = node.Next;
131     }
132     EnsureUnchanged(expectedVersion);
133     // END U3a
134 }
```

Obrázok 6 Zastavenie: Iterátor už našiel prvý uzol; zvýraznené `yield return` ho práve poskytne volajúcemu.

4. Otvorte **Debug > Windows > Locals**. Rozbaľte **node**: Value je **1**, Previous je **null** a Next smeruje na ďalší uzol. Parameter `expectedVersion` má hodnotu **3**, pretože boli vložené tri položky.



Name	Value	Type
> this	{PlaylistLab.Core.Collections.DoublyLinkedList<int>}	PlaylistLab.Core.Collections.D...
> expectedVersion	3	int
> node	{PlaylistLab.Core.Collections.DoublyLinkedListNode<int>}	PlaylistLab.Core.Collections.D...
> Next	{PlaylistLab.Core.Collections.DoublyLinkedListNode<int>}	PlaylistLab.Core.Collections.D...
> Previous	null	PlaylistLab.Core.Collections.D...
> Value	1	int
> Type variables		

Obrázok 7 Premenné: Hodnoty patria prvému zastaveniu. Locals sa aktualizuje pri ďalšom kroku programu.

5. Stlačte **F5 Continue**. Pri ďalšom zastavení na tom istom `yield` má `node.Value` hodnotu **2**. Iterátor pokračoval za predchádzajúcim `yield` a posunul sa cez `Next`. **F10 Step Over** použite na krokovanie jednotlivých príkazov; `yield` vracia riadenie volajúcemu a pokračuje pri ďalšom `MoveNext`. Okno **Debug > Windows > Watch > Watch 1** umožní sledovať vlastný výraz, napríklad `node.Value`.
6. Odstráňte breakpoint opätovným kliknutím alebo **F9** a cez **F5** nechajte blok dobehnúť. Ukončenie ladenia je **Debug > Stop Debugging** alebo **Shift+F5**. Vráťte argument na **--basic**.

Prvý priechod spúšťa pripravené `ToArray`, ktoré tiež opakovane volá `MoveNext`. Neskôr blok ukáže aj priame použitie enumerátora. V Locals sledujte iba hodnoty; nespúšťajte ďalšie `MoveNext` z Watch, aby ste pokus neposunuli.

Zdroj: [krokovanie a hodnoty premenných vo Visual Studiu](#).

U4 Udalosti a odberatelia

50–60 minút

Súbory: Core/Collections/DoublyLinkedList.cs, pripravené CollectionChangedEventArgs.cs a blok U4 v App/Program.cs.

Oznámte úspešnú zmenu

Doplňte telo OnChanged v TODO U4. Vytvorte Changed cez ?.Invoke. Ako sender odovzdajte this. Vytvorte CollectionChangedEventArgs s kind, item a aktuálnym Count.

Metódy AddFirst, AddLast, RemoveFirst a RemoveLast volajú OnChanged **po dokončení zmeny**. OnChanged nevolajte druhýkrát z aplikácie. Pri odmietnutej operácii sa udalosť nevyvolá. Bez odberateľa operácia normálne uspeje.

V starteri je telo OnChanged zámerne bez oznámenia, aby sa dali ostatné bloky skúšať skôr. Riadok _ = Changed pri doplnení nahradzte. Deklarácia udalosti a údaje o zmene sú pripravené.

Prompt: „Doplňte iba telo OnChanged v U4. Použite Changed?.Invoke(this, nové údaje udalosti). Vysvetlite rozdiel medzi event a verejným delegátom a overte, že zlyhanie operácie nevyvolá udalosť. Ostatné metódy ani testy nemeňte.“

Preskúmajte dvoch odberateľov

Blok U4 má pripravené počítadlo a históriu. Oba prihlási cez +=. AddFirst upozorní oboch. Potom odhlási históriu cez -= a RemoveLast upozorní už iba počítadlo.

Očakávajte **počítadlo 2 a históriu 1**. Jediný záznam histórie je **Added: 10, počet 1**. Experimenty robte v aplikácii, dodané testy zachovajte.

Z nasledujúcich pokusov si vyberte jeden; ostatné môžete vyskúšať neskôr.

1. Presuňte odhlásenie až za RemoveLast a predpovedzte počet záznamov histórie. Očakávajte 2. Potom zmenu vráťte.
2. Pridajte do U4 tretieho odberateľa, ktorý vypíše položku a počet po zmene. Použite uloženú premennú delegáta a na konci ho odhláste.
3. V samostatnom krátkom pokuse naplňte kolekciu s kapacitou 1 a skúste pridať ďalší prvok. Pri odmietnutí sa počet oznámení nezvýši.

Pri odhlásení použite **tú istú uloženú inštanciu delegáta**. Opakovanie textu novej lambdy pri -= nevytvorí automaticky rovnakého odberateľa. Udalosť z aplikácie nemôžete priamo vyvolať ani prepísať priradením =.

Nullable operátor v užitočnej situácii

Changed je nullable, pretože môže nemať žiadnych odberateľov. ?.Invoke pri null volanie preskočí.

Operátor ! by stav počas behu neopravil. Keď vypisujete voliteľný Album skladby, použite ?? s textom Bez albumu.

Kontrolný bod: U4_ testy sú zelené. Overte vlastného odberateľa a vysvetlite, prečo údaje udalosti obsahujú počet až po operácii.

U5 Lenivý filter a lambda

60–75 minút

Súbor: Core/Collections/DoublyLinkedList.cs. V U5 doplňte FilterIterator. Verejná metóda WhereMatches a overenie null predikátu sú pripravené.

Vytvorte filter bez okamžitého vykonania

Pomocou foreach prejdite this a pre položky vyhovujúce predicate použite yield return. Zachovajte poradie aj opakovania. Nevytvárajte pole ani List. Parameter **Func<T, bool>** je delegát: prijme položku typu T a vráti výsledok podmienky. Lambda poskytne konkrétnu podmienku.

```
var query = playlist.WhereMatches(t => t.DurationSeconds <= limit);
```

Tento riadok vytvorí postupnosť. Podmienka sa vykonáva až pri foreach, ToArray alebo ToList. Filter môže čítať premennú limit z okolitého kódu; jej neskoršia zmena ovplyvní ďalší priechod.

Prompt: „Doplňte iba U5 FilterIterator cez foreach a yield return. Zachovajte poradie aj opakovania. Vysvetlite Func<T, bool>, odložené vykonanie a zachytenie limit. Pripravené metódy a testy nemeňte.“

Najprv predpovedzte potom spustite

V App spustite **--unit U5**. Ukážka vytvorí playlist T03, T02, T01, premennú limit 180 a počítadlo volaní predikátu. Pred spustením odpovedzte: koľkokrát sa podmienka vykoná pri vytvorení query a čo sa zmení po nastavení limit na 100?

Očakávajte **0 volaní** pri vytvorení query, potom **T03, T01 a 3 volania** pri limite 180. Pri ďalšom priechode po zmene limitu na 100 očakávajte **T03 a spolu 6 volaní**. Pôvodný výsledok snapshot z ToList si ponechá T03 a T01.

ToList vytvorí zoznam referencií, nie hlboké kópie skladieb. Track je v tomto cvičení nemenný. V menu neskôr vyskúšate filter SEKUNDY, ktorý len vypisuje výsledok a skladby z playlistu neodoberá.

Súvis s LINQ: Podobný princíp využíva štandardný filter Where. LINQ bude témou ďalších cvičení; teraz ho nemusíte skúšať ani meniť pripravené volania. ToList tu slúži iba na porovnanie lenivej postupnosti s uchovaným výsledkom.

Pripravený register má where T : class, IIdentifiable a TryGet s NotNullWhen(true). Register vyžaduje jedinečné ID, playlist povoľuje opakovania. V úspešnej vetve TryGet kompilátor pozná nenulový výsledok.

Kontrolný bod: U5_ testy sú zelené. Všetkých 49 dodaných testov teraz prechádza a --basic hlási 0 nedokončených blokov a 0 neočakávaných chýb. Pokračujte vlastnou funkciou.

Váš playlist a záverečné overenie

75–95 minút • vlastné údaje a nová funkcia patria do základu

Najprv overte spoločný scenár

Spustíte App s **--interactive**. Vo Visual Studiu zmeňte argument v debug launch profile; v termináli použite:

```
dotnet run --project PlaylistLab.App -- --interactive
```

V otvorenom konzolovom menu píšete príkazy po jednom a potvrdíte Enter. **add ID** pridá skladbu, **list** vypíše poradie, **next** odoberie prvú skladbu, **reverse** vypíše poradie odzadu a **exit** ukončí menu. Zadajte add T01, add T02, add T01 a list. Poradie je **T01, T02, T01**. add T04 sa pri kapacite 3 odmietne bez oznámenia. next odoberie prvé T01. Potom add T03 vytvorí **T02, T01, T03** a reverse vypíše **T03, T01, T02**. Skúste add T99 a next na prázdnom playliste. Menu sa nesmie ukončiť chybou.

Zvoľte vlastný účel

Vyberte si večierok, cestu vlakom alebo tréning. Zvoľte tri vlastné skladby, ich ID a kladné trvanie. Nemusíte poskytovať hudobné súbory. V **InteractivePlayer.cs** nahraďte iba vytvorenie catalog z SampleData vlastným ItemCatalog<Track> a tromi volaniami Register. Pridajte using PlaylistLab.Core.Services. Kapacitu playlistu ponechajte 3.

Nájdite riadok var catalog = SampleData.CreateCatalog(); a nahraďte ho týmto vzorom. Názvy, interpretov a kladné trvanie v sekundách zmeňte podľa seba. ID P01 až P03 môžete ponechať.

```
var catalog = new ItemCatalog<Track>();
catalog.Register(new Track("P01", "Moja prvá skladba", "Môj interpret", 180));
catalog.Register(new Track("P02", "Moja druhá skladba", "Môj interpret", 120));
catalog.Register(new Track("P03", "Moja tretia skladba", "Môj interpret", 240));
```

Použite Build a znovu spustíte --interactive. V pôvodných kontrolných blokoch zostávajú T01 až T04, vo Vašom menu používajte P01 až P03. Stav playlistu sa po ukončení programu neukladá.

Zadajte **catalog**, **add P01**, **add P02**, **add P01**, **list**. Duplicita v playliste je platná. Pred **reverse** predpovedzte opačné poradie. **add P03** sa pri plnom playliste odmietne; po **next** uspeje. Vyskúšajte **add P99** a odoberanie až do prázdneho playlistu. Pri odmietnutí sa nezvýši počet oznámení Changed.

Ak máte suseda, dajte mu na dve minúty rolu používateľa: nech zadá jednu hraničnú požiadavku a pred výsledkom ju spolu vysvetlite. Samostatne môžete vykonať rovnaký pokus. Príkaz **filter 180** vypíše skladby s trvaním najviac 180 sekúnd. Overte, že následný list stále obsahuje celý playlist.

Samotná zmena skladieb nestačí. Na ďalšej strane si vyberte vlastnú funkciu, navrhnete jej správanie a pridajte testy. Čas 75–95 minút patrí obom týmto častiam.

Vlastná funkcia a vlastné testy

Vyberte si jednu možnosť a dokončite ju do 95. minúty. O výslednom návrhu rozhodujete Vy; AI asistent môže pomôcť. Funkcia musí používať Vašu knižnicu, mať dokumentovaný kontrakt a byť dostupná v menu.

Možnosť A Celkové trvanie playlistu

V Core/Services vytvorte PlaylistStatistics.cs s verejnou statickou triedou PlaylistStatistics a metódou **GetTotalDuration(IEnumerable<Track> tracks)** vracajúcou TimeSpan. Prejdite všetky položky a spočítajte DurationSeconds. Prázdny playlist má nulové trvanie; opakovaná skladba sa počíta za každý výskyt. Obsah, poradie a počet playlistu sa nemenia.

V App/InteractivePlayer.cs pridajte using pre Core.Services a do existujúceho switch nový príkaz **duration**. Vypíšte výsledok s formátom c a doplňte duration aj do textu pomoci menu. Použite pripravený playlist. Pri 180, 120 a znovu 180 sekundách má byť výsledok **00:08:00**.

Možnosť B Nahliadnutie na nasledujúcu skladbu

Do DoublyLinkedList<T> pridajte verejnú metódu **bool TryPeekFirst(out T? item)**. Pri prázdnom zozname vráti false a default; inak true a Value prvého uzla. Položku neodoberá, nemení Count ani verziu a nevyvoláva Changed. Povoľte aj uloženie null hodnotu: prítomnosť položky určuje bool, nie jej hodnota.

Do existujúceho switch v InteractivePlayer.cs pridajte **peek**. Vypíšte názov prvej skladby alebo informáciu o prázdnom playliste a doplňte peek do pomoci menu. Vyskúšajte peek dvakrát a potom list: prvá skladba musí stále zostať na mieste.

Pridajte aspoň tri testy správania

Vo Visual Studiu na Tests použite **Add > Class** a nazvite súbor **OwnFeatureTests.cs**. Vo VS Code či Rideri vytvorte nový .cs súbor v rovnakom projekte. Použite namespace PlaylistLab.Tests, verejnú triedu OwnFeatureTests a xUnit metódy s [Fact]. Vzor testu je v PlaylistTests.cs; pôvodné testy nemeňte.

Pre A overte prázdny playlist, opakovanú skladbu a nezmenené poradie i počet. Pre B overte prázdny zoznam, vrátenú prvú položku a nezmenený stav bez oznámenia udalosti. Testy majú samostatne zostaviť vstup, vykonať operáciu a cez Assert overiť výsledok.

Prompt: „Navrhňte implementáciu mojej zvolenej funkcie a tri xUnit testy podľa kontraktu. Pred kódom vysvetlite prípady, ktoré testy rozlíšia. Použite existujúcu knižnicu a nové OwnFeatureTests.cs; pôvodné testy zachovajte. Zapojte nový príkaz do existujúceho menu.“

Overte že test naozaj odhalí chybu

Dočasne vložte chybu: pri A počítajte každé ID iba raz, pri B odoberte položku namiesto nahliadnutia. Aspoň jeden Váš test musí zlyhať. Potom chybu odstráňte a spustite všetky testy. Pridajte XML summary k novej metóde a prezrite jej dokumentáciu v IntelliSense.

Základ máte hotový, keď prechádza **49 dodaných testov aj aspoň 3 Vaše testy**, --basic hlási 0 a 0 a nový príkaz funguje vo vlastnom playliste. Na CLI použite **dotnet test PlaylistLab.slnx**, vo Visual Studiu Test Explorer a Run All. Zvyšných 5 minút je rezerva. Riešenie ani odpovede neodovzdávate.

Dobrovoľná NuGet tabuľka

Po dokončení vlastnej funkcie alebo po hodine. Filter z U5 už patrí do základu, tu meníte iba zobrazenie údajov.

Vzhľad aplikácie cez Spectre Console

Vo **Visual Studiu 2026** vyberte App a **Project > Manage NuGet Packages**. Na Browse zvolíte nuget.org, vyhľadajte Spectre.Console, nastavte verziu **0.57.2** a kliknite Install. Potvrďte prípadné licenčné okno. Skontrolujte Dependencies > Packages. Do Core balíček nepatrí.

Na **macOS a Linuxe** použite z koreňa solution rovnaké CLI, dostupné aj v termináli Visual Studia:

```
cd PlaylistLab.App
dotnet add package Spectre.Console --version 0.57.2
cd ..
```

PlaylistView.cs nahraďte vzorom z Pomoci pre NuGet tabuľku. Text ochráni názvy ako [LIVE] pred interpretáciou markup. Album zobrazte cez ?? Bez albumu. V menu porovnajte výpis pred úpravou a po nej.

Prompt: „Nahraďte iba PlaylistView.cs tabuľkou cez Spectre.Console 0.57.2. Údaje zobrazujte cez Text, chýbajúci album cez ?? a UsesSpectre nastavte na true. Core a testy nemeňte.“

Pri základnom AddLast je pripravený XML komentár. Môžete spresniť summary, spustiť Build a overiť XML pri DLL aj IntelliSense. Po doplnení tabuľky blok --unit U6 aj celý program bez argumentov hlásia 0 a 0. U6 je iba dobrovoľný výpis, nemá ďalšie povinné testy.

Ďalšie možnosti podľa záujmu

Vytvorte Parcel s nemenným Id a hmotnosťou a použite tú istú generickú knižnicu pre zásielky. Alebo dokončite druhú vlastnú funkciu. Lokálne balenie knižnice je v poslednej kapitole. Tieto časti sa od všetkých študentov v 100 minútach nevyžadujú.

Pomoc pre operácie nad uzlami

Najprv použite slovnú nápovedu. Ak ste bez posunu, vložte krátky vzor do **existujúceho tela metódy** medzi BEGIN a END. Vzory nevytvárajú ďalšie triedy. Pred spustením vysvetlite zmenu oboch koncov a potom spustite U2_ testy.

U2a Pridanie na koniec

Nový uzol pozná pôvodný posledný cez Previous. Pôvodný posledný musí získať Next. Pri prvom vložení ukazujú _first aj _last na ten istý uzol.

```
EnsureSpace();
var node = new DoublyLinkedListNode<T>(item) { Previous = _last };
if (_last is null) _first = node;
else _last.Next = node;
_last = node;
Count++;
_version++;
OnChanged(CollectionChangeKind.Added, item);
```

U2b Odobratie zo začiatku

Uchovajte odoberaný uzol, aby ste vedeli vrátiť jeho hodnotu. ?? throw bezpečne rozlíši prázdny zoznam ešte pred zápisom. Posledná položka si vyžaduje opravu oboch koncov.

```
var node = _first ?? throw new InvalidOperationException("Zoznam je prázdny.");
_first = node.Next;
if (_first is null) _last = null;
else _first.Previous = null;
node.Next = null;
Count--;
_version++;
OnChanged(CollectionChangeKind.Removed, node.Value);
return node.Value;
```

Ako čítať zlyhanie testu

NotImplementedException znamená nedoplnené telo. Ak Expected nesúhlasí s Actual, prečítajte kontrakt a vstupy testu. Pri U4 je zatiaľ bežný nesprávny počet oznámení, pretože jeho pripravené telo nič neoznamuje.

Pri chybe prekladu otvorte uvedený súbor a riadok v Error List; testy spúšťajte až po úspešnom Build. Pri chýbajúcom balíčku použite dotnet restore PlaylistLab.slnx. Ak sú hneď všetky testy zelené, skontrolujte, ktorý variant zdrojov máte otvorený.

Nevymieňajte celý DoublyLinkedList za vstavanú kolekciu: cieľom U2 a U3 je porozumieť uzlom a iterátoru. V bežnej aplikácii by ste vhodnú existujúcu kolekciu zvažili ako prvú.

Pomoc pre iterátory

U3a Dopredný iterátor

Vzor vložte do EnumerateForward. expectedVersion je už pripravený parameter. EnsureUnchanged na konci zachytí aj zmenu pred ukončením priechodu.

```
var node = _first;
while (node is not null)
{
    EnsureUnchanged(expectedVersion);
    yield return node.Value;
    node = node.Next;
}
EnsureUnchanged(expectedVersion);
```

U3b Spätný iterátor

Vzor patrí do EnumerateReverse. Nie je potrebné pole ani prevrátenie skopírovaného zoznamu.

```
var node = _last;
while (node is not null)
{
    EnsureUnchanged(expectedVersion);
    yield return node.Value;
    node = node.Previous;
}
EnsureUnchanged(expectedVersion);
```

Krátky ručný priechod

Nasledujúci samostatný pokus v Program.cs použite až po doplnení U3. Vytvorí samostatný enumerátor a using ho po skončení uvoľní. Pri Debug krokovani sledujte presun medzi volajúcim a yield.

```
var numbers = new DoublyLinkedList<int>();
numbers.AddFirst(2);
numbers.AddFirst(1);
using var iterator = numbers.GetEnumerator();
while (iterator.MoveNext())
    Console.WriteLine(iterator.Current);
```

Výsledok je 1 a 2. MoveNext vracia bool, Current aktuálnu položku. GetEnumerator sám ešte neposkytol prvú hodnotu. Typ IEnumerator predstavuje stav konkrétneho priechodu; IEnumerable umožňuje nový priechod získať.

Pomoc pre udalosti a filter

U4 Oznámenie zmeny

Vložte iba telo OnChanged. Samotná táto metóda nezvyšuje Count ani nemení uzly, tie už zmenila príslušná operácia.

```
Changed?.Invoke(this, new CollectionChangedEventArgs<T>(kind, item, Count));
```

Odberateľa môžete uložiť ako EventHandler<CollectionChangedEventArgs<int>>. Do += a -= vložte rovnakú premennú. Trieda zoznamu je sealed, preto v tomto návrhu nepotrebujeme protected virtual OnChanged.

U5 Filter

Vzor patrí do FilterIterator. foreach používa váš dopredný enumerátor z U3. Vrátený filter sa vyhodnocuje pri každom novom priechode znovu.

```
foreach (T item in this)
    if (predicate(item))
        yield return item;
```

Prečo nevracať hneď ToList: Úloha skúma odložené vykonanie, zachytenú premennú a rozdiel oproti materializovanému zoznamu. Ak potrebujete stabilný zoznam výsledkov, ToList zavolá používateľ API.

Pomoc pre NuGet tabuľku

Po nainštalovaní Spectre.Console 0.57.2 do App nahraďte **celý obsah App/PlaylistView.cs** nasledujúcim vzorom. Nie je to ďalší súbor vedľa pôvodnej triedy. Core na balíček nemá referenciu.

```
using PlaylistLab.Core.Models;
using Spectre.Console;

namespace PlaylistLab.App;

internal static class PlaylistView
{
    internal static bool UsesSpectre => true;

    internal static void Print(IEnumerable<Track> tracks)
    {
        var table = new Table();
        table.AddColumn("ID");
        table.AddColumn("Skladba");
        table.AddColumn("Interpret");
        table.AddColumn("Album");
        table.AddColumn("Trvanie");
        foreach (var track in tracks)
            table.AddRow(new Text(track.Id), new Text(track.Title),
                new Text(track.Artist), new Text(track.Album ?? "Bez albumu"),
                new Text(TimeSpan.FromSeconds(track.DurationSeconds).ToString("c")));
        AnsiConsole.Write(table);
    }
}
```

Text zobrazuje údaje bez interpretácie markup značiek. ToString("c") zobrazuje trvanie aj nad jednu hodinu. UsesSpectre umožní pripravenému kontrolnému bloku rozlíšiť základný výpis od dokončenej tabuľky.

Čo overiť po vložení

Spustite Build, všetky testy a blok U6, ktorý zobrazí štyri pôvodné skladby. Blok U5 naďalej overuje filter s výsledkom T03 a T01, pri oboch Album Bez albumu. Vyskúšajte vo vlastných ukázkových údajoch názov obsahujúci [LIVE]. Má sa vypísať doslova.

Súvis s praxou: [LINQ a odložené vykonanie](#) a [výber hotovej kolekcie](#).

Zdroj: [Spectre.Console tabuľky](#). Ďalšie vysvetlenie: [iterátory yield](#), [nullable atribúty](#) a [XML dokumentácia vo Visual Studiu](#).

Rozšírenie s vlastným NuGet balíčkom

Dobrovoľné rozšírenie po dokončení základov, vhodné aj na pokračovanie po hodine.

Zabaľte knižnicu

Záložný starter už obsahuje metadáta balíčka. Vo vlastnom Core.csproj najprv pridajte do PropertyGroup PackageId PlaylistLab.Core a Version 1.0.0. XML dokumentácia bola zapnutá v U1. Príkaz pack vytvorí balíček s DLL a XML; ďalšie metadáta a README sú vhodným pokračovaním. V existujúcom PropertyGroup vlastného projektu budú tieto dva nové riadky:

```
<PackageId>PlaylistLab.Core</PackageId>
<Version>1.0.0</Version>
```

Potom v koreňovom priečinku vykonajte:

```
dotnet pack PlaylistLab.Core/PlaylistLab.Core.csproj -c Release -o ./local-feed
dotnet new console -n PackageConsumer -f net10.0
cd PackageConsumer
dotnet add package PlaylistLab.Core --version 1.0.0 --source ../local-feed
cd ..
```

PackageConsumer je nová aplikácia, ktorá na Core nemá ProjectReference. Vyhľadajte jej PackageReference. V tomto prípade vlastná knižnica nemá externé balíkové závislosti.

V jej Program.cs použite:

```
using PlaylistLab.Core.Collections;

var numbers = new DoublyLinkedList<int>(3);
numbers.AddLast(10);
numbers.AddLast(20);
Console.WriteLine(string.Join(", ", numbers.Reverse()));
```

Po dotnet run --project PackageConsumer očakávajte **20, 10**. Podržte myš nad AddLast a skontrolujte dokumentačný komentár. V balíčku .nupkg môžete cez ZIP prehliadač nájsť DLL aj XML. Vlastné Core.csproj nemá automaticky nastavený README ani autora balíčka.

Malý experiment s verziou

Zmeňte dokumentáciu alebo pridajte vlastný verejný člen. Verziu v Core zvýšte na 1.0.1, znovu vykonajte pack a v aplikácii nainštalujte 1.0.1. Overtite nový člen alebo IntelliSense. Pri zmene balíčka používajte novú verziu, aby vás neplietla lokálna cache.

Prompt: „Pomôžte mi zabaliť dokončený PlaylistLab.Core do lokálneho NuGet balíčka a použiť ho v novej Console App cez PackageReference. Použite lokálny priečinok local-feed a vysvetlite rozdiel oproti ProjectReference. Zachovajte existujúci starter a testy.“

Balíček stačí používať lokálne. Na cvičení nepotrebuje účet na nuget.org ani publikovanie na verejný server.

Zdroj: [vytvorenie balíčka cez dotnet CLI](#) a [lokálne NuGet zdroje](#).

Oficiálne postupy pre prostredie: [vlastná knižnica](#), [NuGet vo Visual Studiu 2026](#), [debug launch profily](#), [projekty vo VS Code](#) a [Run konfigurácia Rider](#).